

Course Code: **IDC 222**



Scientific Computing & Data Visualization

Reji J

School of Data Science

IISER Thiruvananthapuram

Switch Statements



switch statement

- A **switch statement** helps us choose **one option from many choices**.
- If none of them are match, then **default case** will execute.

Example: Simple Calculator

- Let's say we're building a calculator program where the user picks what they want to do:

Press 1 → Do Addition

Press 2 → Do Subtraction

Press 3 → Do Multiplication

Press 4 → Do Division

switch statement checks which number the user pressed and performs only that operation.



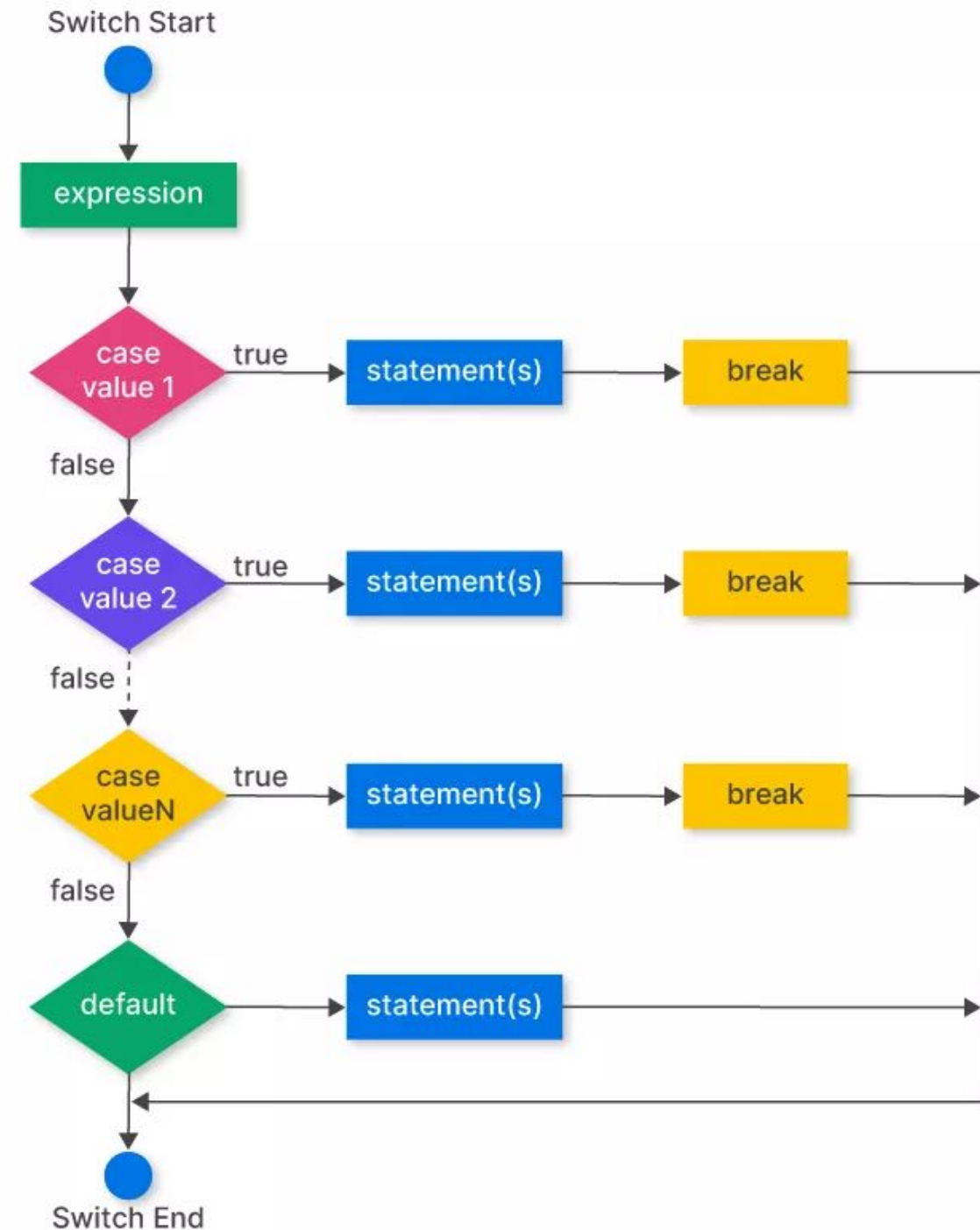
switch statement

Syntax:

```
switch(expression)
{
    case value 1:
        //code to be executed;
        break;
    case value 2:
        //code to be executed;
        break;

    default:
        //code to be executed if all cases
        are not matched;

    break;
}
```



```

#include <iostream>
using namespace std;

int main()
{
    char x = 'A';

    switch (x) {
        case 'A':
            cout << "Choice is A";
            break;
        case 'B':
            cout << "Choice is B";
            break;
        case 'C':
            cout << "Choice is C";
            break;
        default:
            cout << "Choice other than A, B and C";
            break;
    }
    return 0;
}

```

Important Rule

Your **choice** variable can only be:

int (integer) - Example: 1, 2, 3 etc
char (character) - Example: 'A', 'B', '+' etc

Other Data types are,

✗ NOT allowed: float, double, or string



WAP to print whether the entered character is a vowel or a consonant

```
#include <iostream>
using namespace std;
int main()
{
    char ch;
    cout << "Enter any Alphabet\n";
    cin >> ch;

    switch(ch)
    {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
        case 'A':
        case 'E':
        case 'I':
        case 'O':
        case 'U':

            cout << ch << " is a vowel";

            break;
        default:

            cout<<ch<<" is a consonant";

            break;
    }
    return 0;
}
```



Rules of the switch case statement

1. In a switch statement, the “case value” must be of “char” and “int” type.
2. There can be one or N number of cases.
3. The values in the case must be unique.
4. Each statement of the case can have a break statement. It is optional.
5. The default Statement is also optional.



Implement a Physics calculator in C++:

1. **Force** ($F = m \times a$)
2. **Work** ($W = F \times d \times \cos(\theta)$)


```

#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int choice;
    cout << "Physics Calculator Menu:\n";
    cout << "1.Calculate Force\n";
    cout << "2.Calculate Work\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice)
    {
        case 1:
        {
            double mass, acceleration, force;
            cout << "Enter mass (kg): ";
            cin >> mass;
            cout << "Enter acceleration (m/s^2): ";
            cin >> acceleration;
            force = mass * acceleration;
            cout << "Force = " << force << " N" << endl;
            break;
        }
    }
}

```

```

case 2:
{
    double force, distance, theta, work;
    cout << "Enter force (N): ";
    cin >> force;
    cout << "Enter distance (m): ";
    cin >> distance;
    cout << "Enter angle (theta in degrees): ";
    cin >> theta;
    theta = theta * M_PI / 180;
    work = force * distance * cos(theta);
    cout << "Work = " << work << " J" << endl;
    break;
}

default:
    cout << "Invalid choice!";
}

return 0;
}

```

Write a Menu driven C++ program for a basic calculator that performs arithmetic operations (addition, subtraction, multiplication, and division).

The program should prompt the user to enter two numbers and an operator (+, -, *, /).

Implement the calculator using a switch-case statement and display the result in the format

<number1> <operator> <number2> = <result>.

```

#include <iostream>
using namespace std;

int main()
{
    float num1, num2, result;
    char operation;
    char choice;

    cout << "\n--- Calculator Menu ---" << endl;
    cout << "Addition (+)" << endl;
    cout << "Subtraction (-)" << endl;
    cout << "Multiplication (*)" << endl;
    cout << "Division (/)" << endl;
    cout << "Exit" << endl;
    cout << "Enter your choice (+, -, *, /): ";
    cin >> choice;

    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    switch (choice)
    {
        case '+':

            result = num1 + num2;
            break;

        case '-':

            result = num1 - num2;
            break;

```

```

        case '*':

            result = num1 * num2;
            break;

        case '/':

            if (num2 != 0) {
                result = num1 / num2;
            } else {
                cout << "Error: Cannot divide by zero." << endl;
            }
            break;

        default:
            cout << "Invalid choice." << endl;
    }
    cout << num1 << " " << choice << " " << num2 << " = " << result;

    return 0;
}

```

switch statement v/s if-else if statement

- Both are multiple branch statements.
- Both have an optional statement (**else statement** and **default statement**), which work in the same fashion.
- The expressions associated with **if** statements are **relational (or logical)**, where **integer or character expressions** are used for the **switch** statement.
- The **break** statement is a must in **switch** statement, if control should be transferred outside switch after executing a case statement. It is not needed in if-else if statement
- The statement sequence associated with each case of switch statement does not require a pair of braces (); but it is needed in if-else if statement to make them a compound statement.
- Only equality checking of two values are done in switch statement; but any kind of comparison can be made for condition checking in if else if statement



Looping Statements



Looping Statements

- **Looping statements** allow us to **repeat** a set of instructions **multiple times** automatically.
- It is also called **iteration statements**.
- The loop keeps running **until a condition becomes false**.

C++ provides 3 types of loops:

1. **for** loop
2. **while** loop
3. **do-while** loop



How loop works?

How Do Loops Work?

- Loops repeat a set of instructions **based on a condition**.
- To control the loop, we use a **special variable** called the **loop control variable**.

The 3 Parts of Every Loop

- Every loop has **3 important expressions** that work together:

1) Initialization Expression

- **Gives the starting value** to the loop control variable
- Happens **only once** at the beginning
- Example: **i = 1** (starts counting from 1)



Contd...

2) Test Expression (Condition)

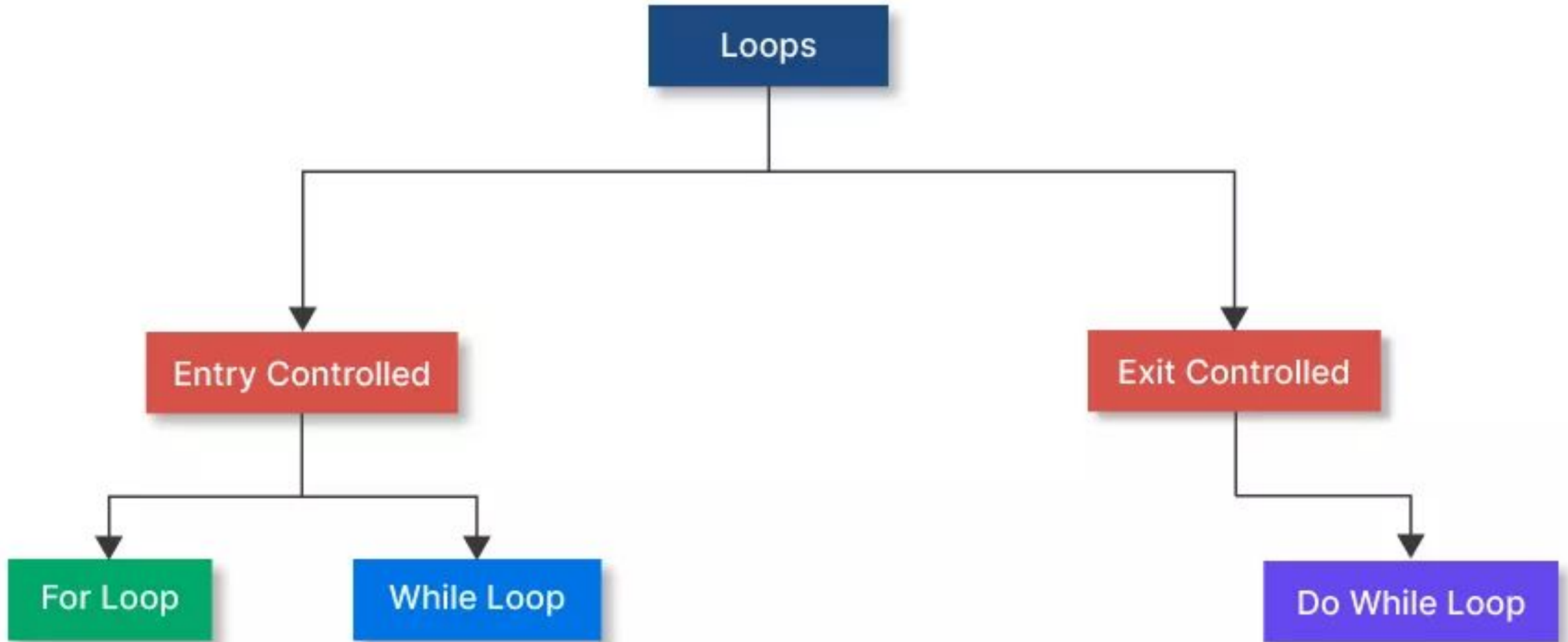
- **Checks** if the loop should continue or stop
- Uses relational operators like `<`, `>`, `<=`, `>=`, `==`, `!=`
- If **TRUE** → Loop body executes
- If **FALSE** → Loop stops
- Example: `i <= 10` (keep going while `i` is less than or equal to `10`)

3) Update Expression

- **Changes** the value of the loop control variable after each repetition
- Usually increases or decreases the variable
- Example: `i++` (increase `i` by `1`)



Types of Loops in C



Classification of Loops

1) Entry Controlled Loop ("Check first, then enter")

- **Condition is checked BEFORE** entering the loop
- If condition is TRUE → Execute loop body
- If condition is FALSE → Skip the loop entirely
- Loop may not run even once if condition is false from the start

Examples: for loop, while loop

2) Exit Controlled Loop ("Enter first, then check")

- Loop body **executes at least once** without checking condition
- **After first execution**, condition is checked
- If condition is TRUE → Repeat
- If condition is FALSE → Stop

Examples: do-while loop



For Loop

- A for loop repeats a block of code multiple times until a condition becomes false.
- **for** loop is an entry-controlled loop. All its loop control elements are gathered at one place.
- In general, we should use **for** loop when we know the exact loop count.

Syntax:

```
for (initialization expression; test condition; update expression)
{
    //body of the loop
}
```

Example: Print numbers from 1 to 10

```
for ( i = 1 ; i <= 10 ; i++ )
{
    cout << i ;
}
```



Flowchart of for Loop

Example:

print numbers in reverse order

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{
```

```
    for(int i = 10; i >= 1; i--)
```

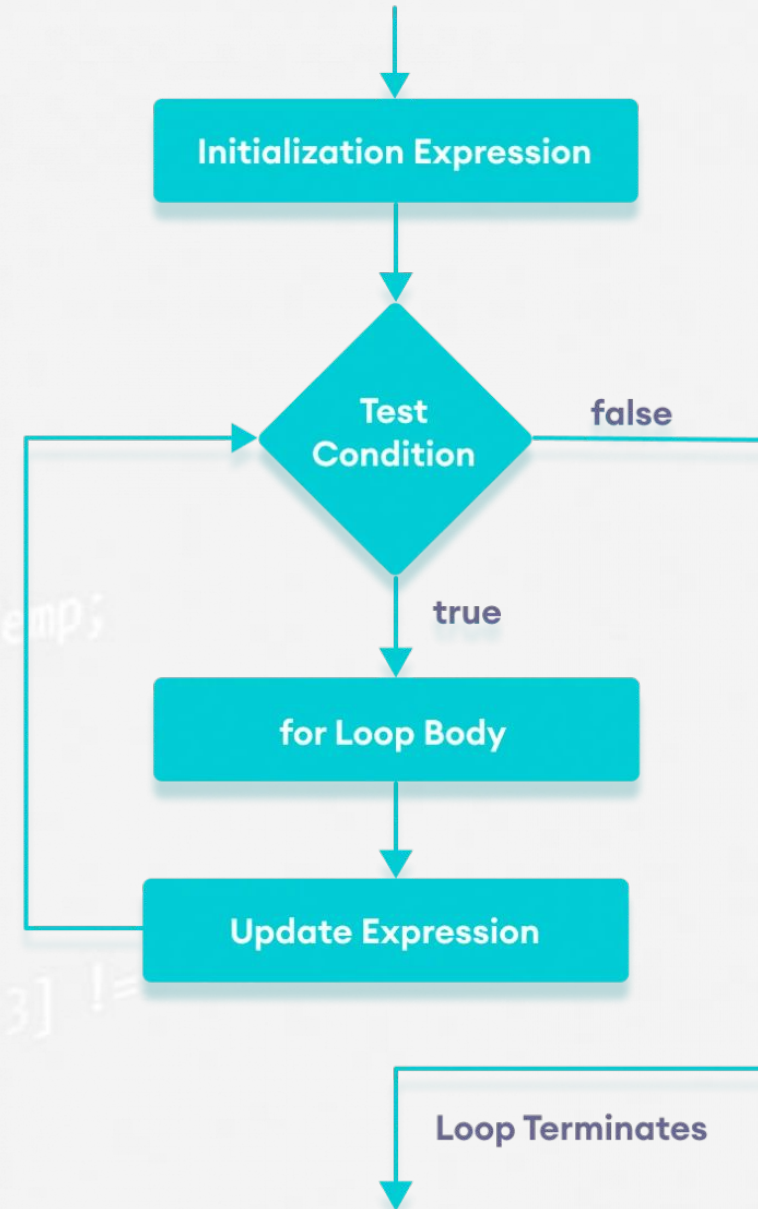
```
    {
```

```
        cout << i << endl;
```

```
    }
```

```
    return 0;
```

```
}
```



While Loop

- The **while** loop is also an **entry-controlled loop**.
- Use **while loop** when you **don't know exactly how many times** the loop should run.
- The number of repetitions **depends on user input** or some condition
- Loop count is **not fixed** in advance

Syntax:

```
initialization expression
```

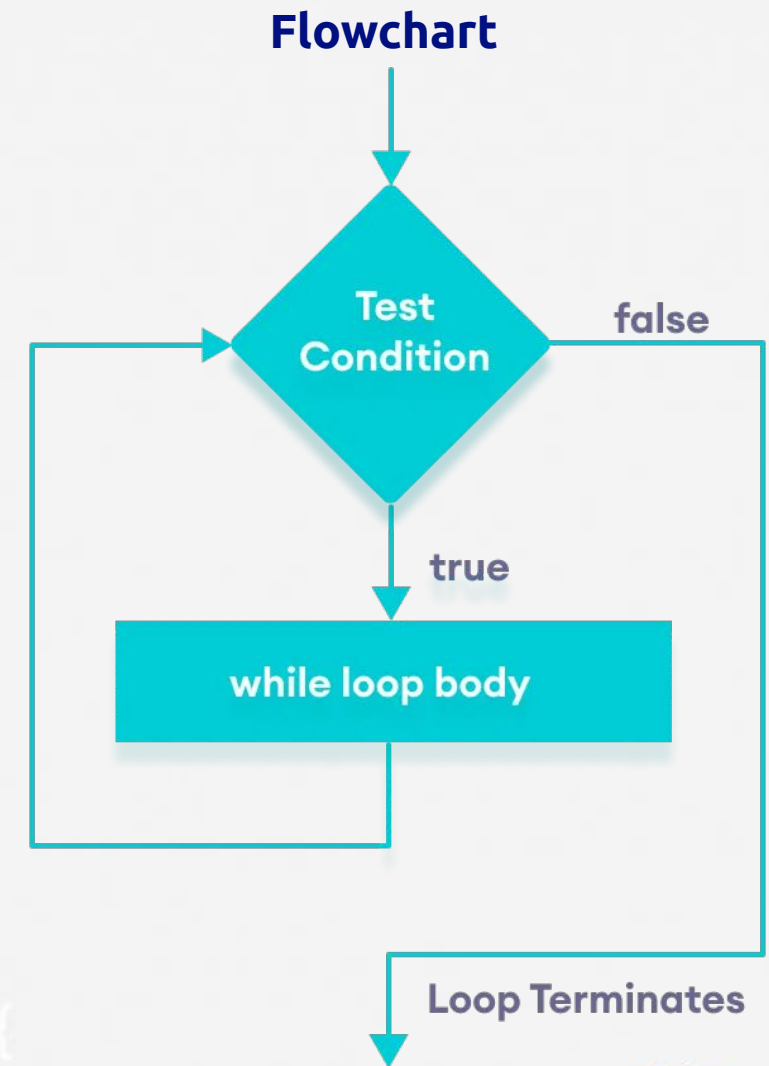
```
while (condition)
```

```
{
```

```
    //statements
```

```
    update expression
```

```
}
```



Example:

Print even numbers between 1 and 20 using while loop

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{
```

```
    int i = 2;
```

```
    while (i <= 20)
```

```
    {
```

```
        cout << i << endl;
```

```
        i += 2;
```

```
    }
```

```
    return 0;
```

```
}
```



Do While Loop

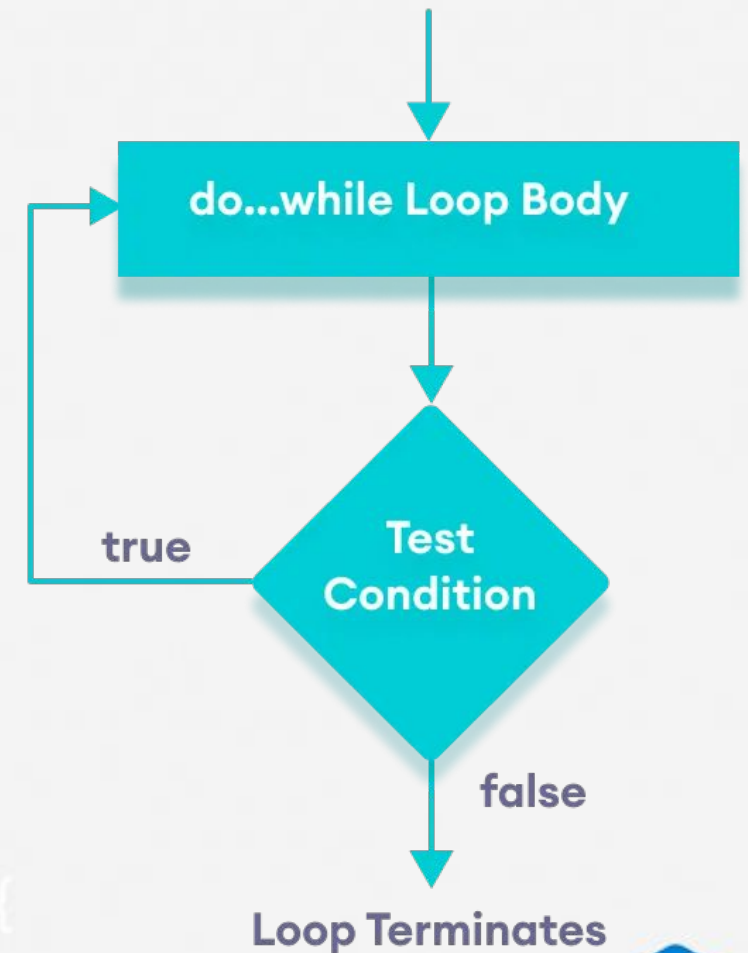
- The **do while** loop is also an **exit-controlled loop**.
- It is used when you want to execute statements **at least once**, regardless of the **condition**.
- **do while** will execute statements **at least once** without checking the condition.

Syntax:

```
do
{
    //statements
    update expression
}while(condition);
```

! **semicolon** after **while** (condition);

Flowchart



Example: Display a menu and get user choice

```
int choice;

do
{
    printf("\n=== MENU ===\n");
    printf("1. Add\n");
    printf("2. Subtract\n");
    printf("3. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    // Process choice...

} while (choice != 3);
```

Why do-while?

- Menu must be shown **at least once**
- User can't make a choice without **seeing the menu first!**





Exercise 1

Write a C++ program to find the sum of the factorial series using a **for loop**.

$$1! + 2! + 3! + \dots + n!$$

Program:

```
#include <iostream>
using namespace std;

int main()
{
    int n, i;
    int f = 1, sum = 0;

    cout << "Number of terms: ";
    cin >> n;

    for (i = 1; i <= n; i++)
    {
        f = f * i;
        sum = sum + f;
    }

    cout << "Factorial series sum : " << sum << endl;
    return 0;
}
```





Exercise 2

Write a C++ programs to find the sum of the following series using loop.

$$a + ar + ar^2 + \dots + ar^{(n-1)}$$

Example Input/Output:

First term : 2

Common ratio : 3

Number of terms : 4

GP series sum = 80

Program:

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    float a, r, sum = 0;
    int n;

    cout << "First term: ";
    cin >> a;
    cout << "Common ratio: ";
    cin >> r;
    cout << "Number of terms: ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        sum = sum + a * pow(r, i);
    }

    cout << "GP series sum = " << sum << endl;
    return 0;
}
```





Exercise 3

Write a C++ program to count the number of digits in a given number using a **while** loop.

Example Input/Output:

```
Enter a number    : 12345
```

```
Number of digits = 5
```

Program:

```
#include <iostream>
using namespace std;

int main()
{
    int n, count = 0;

    cout << "Enter a number: ";
    cin >> n;

    while (n != 0)
    {
        count++;
        n = n / 10;
    }

    cout << "Number of digits = " << count << endl;
    return 0;
}
```





Exercises

Write C++ programs to find the sum of the following series using loops.

1. $1 + 11 + 111 + \dots$ (up to n terms)

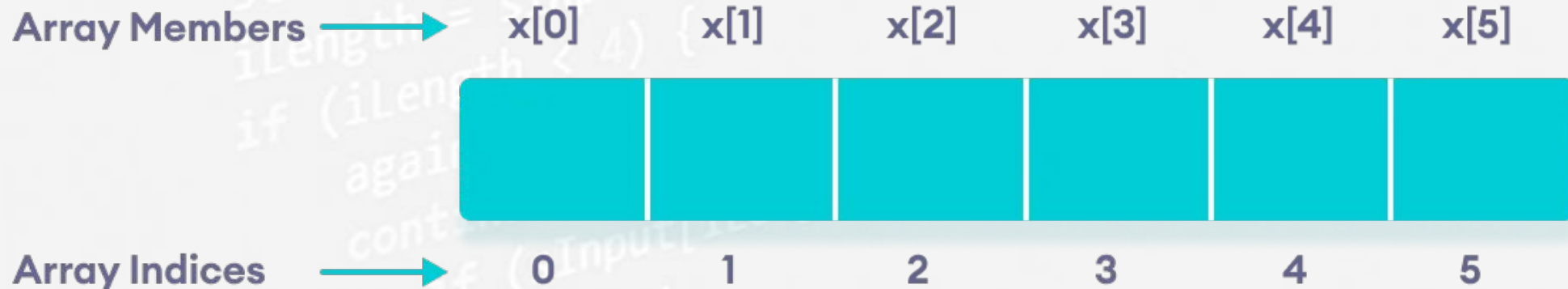
2. $x + x^2 + x^3 + \dots + x^n$

Array



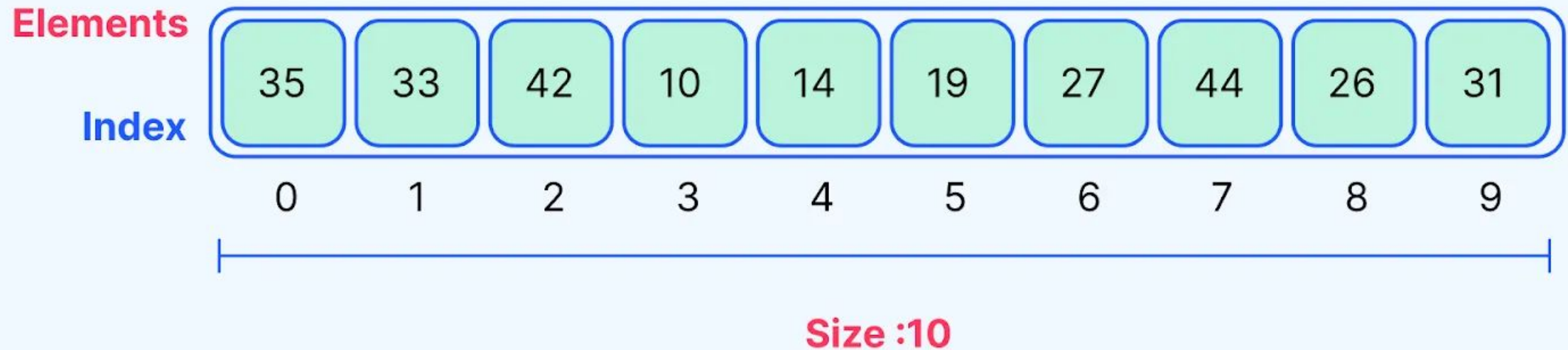
Array

- An array is a collection of elements of the **same data type**.
- Elements are stored in **contiguous memory locations**.
- The **elements of an array** can be accessed using an **index** (starting from 0).
- The data type can be primitive (int, char, float) or user-defined.
- Arrays help in **storing multiple values** in a single variable instead of declaring separate variables for each value.



Representation of an array

Name
Type **array** **[10]** = {35, 33, 42, 10, 14, 19, 27, 44, 26, 31}
Size **Elements**



Contd...

- **Important:** An array can store only one data type at a time.

a

5	6	10	13	56	76	1	2	4	8
---	---	----	----	----	----	---	---	---	---



b

'a'	'b'	'c'	'd'	'e'
-----	-----	-----	-----	-----



c

'a'	'b'	1	5.6	'e'	34	2	3
-----	-----	---	-----	-----	----	---	---



- In array **a**, all stored values are **integers** -  valid.
- In array **b**, all stored values are **characters** -  valid.
- In array **c**, values include **integers**, **floats**, and **characters** -  invalid.
- An array cannot store multiple data types at once.

Array Declaration

Syntax:

```
datatype array_name[size];
```

Example:

```
int age[10];  
char name[100];  
float weight[5];
```

Array Initialization

Example:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Access Array Elements

```
cout << a[0];    // first element  
cout << a[4];    // last element
```



Example: Take Inputs from User and Store Them in an Array

```
#include <iostream>
using namespace std;

int main()
{
    int n, i;
    cout << "Enter the array size:\n";
    cin >> n;
    float a[n];

    cout << "Enter " << n << " elements:\n";
    for (i = 0; i < n; i++)
    {
        cin >> a[i];
    }

    cout << "Array elements are:\n";
    for (i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }
    return 0;
}
```

} // Input

} // Output





Exercise 1

- Write a C++ program to search for an element in an array.

Example Output:

```
Enter the size of the array: 5
```

```
Enter array elements
```

```
11 22 33 44 55
```

```
Enter the element to search: 44
```

```
Element 44 found at position 4
```

Program:

```
#include <iostream>
using namespace std;

int main()
{
    int a[5], key, found = 0;

    cout << "Enter array elements: ";
    for (int i = 0; i < 5; i++)
    {
        cin >> a[i];
    }

    cout << "Enter element to search: ";
    cin >> key;

    for (int i = 0; i < 5; i++)
    {
        if (a[i] == key)
        {
            found = 1;
            break;
        }
    }

    if (found == 1)
        cout << "Element found" << endl;
    else
        cout << "Element not found" << endl;

    return 0;
}
```



String



String

- A string is a collection of characters used to store and manipulate text.
- In C++, strings can be handled in two ways:

1. C-Style String (Character Array)

2. C++ String class (std::string)



1. C-Style String (Character Array)

Declaration



```
char str[20];
```

Initialization



```
char str[] = "Hello";
```

Ends with a
null character `'\0'`

Reading a string



```
cin >> str; // reads single word
```

Printing a string



```
cout << str;
```



Program:

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{
```

```
    char name[20];
```

```
    cout << "Enter name : ";
```

```
    cin.getline(name, 20);
```

```
    cout << name;
```

```
    return 0;
```

```
}
```

What is `cin.getline()`?

- It is used to **read an entire line of text, including spaces**, from standard input.
- It is mainly used with character arrays (C-style strings).

Syntax:

```
cin.getline(string_name, size);
```



2. C++ String class (std::string)

- **std::string** is a built-in class in C++.
- Header file : `#include <string>`

Declaration

`string s;`

Initialization

`string s = "Hello";`

Input & Output

`cin >> s;           // single word`

`getline(cin, s);   // full line`

`cout << s;`



Program:

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string name;
    cout << "Enter name: ";
    getline(cin, name);

    cout << "Name : " << name;
    return 0;
}
```

What is `getline()`?

- is used to read an entire line of text, including spaces, and store it in a C++ string (`std::string`).
- It is part of the C++ string class.

Syntax:

```
getline(cin, string_variable);
```



String Concatenation

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string first, second, result;

    cout << "Enter first string: ";
    getline(cin, first);

    cout << "Enter second string: ";
    getline(cin, second);

    result = first + " " + second;
    cout << "Combined String: " << result;
    cout << "The string length is: " << result.length();

    return 0;
}
```



Function



Function

What is a Function?

- A function is a **block of code** that performs a **specific task**.
- It runs **only when it is called**.
- Every C++ program starts with the **main()** function.

Why Do We Need Functions?

- Break a large program into **smaller parts**
- **Avoid repetition** of code
- Improve **readability** and **maintainability**
- Easier **debugging** and **testing**

Syntax:

```
return_type function_name(parameter_list)
{
    // function body
}
```



Types of Functions in C++

1. Standard Library Functions

- These are **predefined functions in C++**.
- Available through header files like `<cmath>`, `<string>` etc.

Examples:

- `sqrt()` from `<cmath>` - finds square root
- `length()` from `<string>` - find length of the string

2. User-defined Functions

- Functions created by the programmer.
- C++ allows users to define their own functions to perform specific tasks.
- User-defined functions help in **code reusability, readability, and modularity**.



Function Definition (in User-Defined Functions)

- A function must be **declared or defined** before it is used in a program.
- A function definition specifies:
 - the **name** of the function
 - the **return type**
 - the **parameters (if any)**
 - the **body** of the function

Syntax:

```
return_type function_name(parameter_list)
{
    // function body
}
```



Example : Function Definition

```
#include <iostream>
using namespace std;
```

```
int add(int x, int y)
{
    return x + y;
}
```

```
int main()
{
    int result;

    result = add(10, 20);

    cout << "Result is : " << result;
    return 0;
}
```

Called function

Calling function



Example : Area and Perimeter of a Circle using Function

```
#include <iostream>
#include <cmath>
using namespace std;

float computeArea(float r)
{
    return M_PI * r * r;
}

float computePerimeter(float r)
{
    return 2 * M_PI * r;
}

int main()
{
    float radius;

    cout << "Enter radius: ";
    cin >> radius;

    cout << "Area of circle is " << computeArea(radius) << endl;
    cout << "Perimeter of circle is " << computePerimeter(radius) << endl;

    return 0;
}
```



Example : C++ Function can return a String

```
#include <iostream>
#include <string>
using namespace std;

string getWeatherStatus(int temp)
{
    if (temp > 35)
        return "Temperature is hot\n";
    else if (temp >= 20)
        return "Temperature is warm\n";
    else
        return "Temperature is cold\n";
}

int main()
{
    int temperature = 30;
    cout << getWeatherStatus(temperature);
    return 0;
}
```



Exercise :

Given a **set of rainfall measurements** (in millimeters) stored in an array, write a C++ program to:

1. Pass the array to a function
2. Find and return the maximum rainfall value
3. Display the result in the main function

Example Input/Output:

Enter number of values: **6**

Enter rainfall values:

12.5 18.2 15.0 20.4 17.8 16.3

Maximum Rainfall = **20.4 mm**



Program :

```
#include <iostream>
using namespace std;

double findMax(double arr[], int n)
{
    double max = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (arr[i] > max)
        {
            max = arr[i];
        }
    }

    return max;
}
```

```
int main()
{
    int n;
    cout << "Enter number of rainfall readings: ";
    cin >> n;

    double rainfall[n];

    cout << "Enter rainfall values (in mm):\n";
    for (int i = 0; i < n; i++)
    {
        cin >> rainfall[i];
    }

    double maxRainfall = findMax(rainfall, n);

    cout << "Maximum Rainfall = " << maxRainfall << " mm";

    return 0;
}
```



Types of User Defined Functions in C++

1. Function with arguments and a return value

- Receives arguments.
- Returns a value of any valid data type.

2. Function with arguments and no return value

- Receives arguments from the calling function.
- Return type is void.

4. Function with no arguments and a return value

- Does not receive any arguments.
- Returns a value of any data type (not void).

3. Function with no arguments and no return value

- Does not receive any data from the calling function.
- Does not return any value.
- Return type is void.



1. Function with Arguments and a Return Value

```
#include <iostream>
using namespace std;

int add(int x, int y)
{
    return x + y;
}

int main()
{
    int result;
    result = add(10, 20);
    cout << "Result is : " << result << endl;
    return 0;
}
```



2. Function with Arguments and No Return Value

```
#include <iostream>
using namespace std;

void add(int x, int y)
{
    int sum = x + y;
    cout << "Result is : " << sum << endl;
}

int main()
{
    add(10, 20);
    return 0;
}
```



3. Function with No Arguments and a Return Value

```
#include <iostream>
using namespace std;

int add() {
    int x = 10, y = 20;
    return x + y;
}

int main()
{
    int result;
    result = add();
    cout << "Result is : " << result << endl;
    return 0;
}
```



4. Function with No Arguments and No Return Value

```
#include <iostream>
using namespace std;

void add() {
    int x = 10, y = 20;
    int sum = x + y;
    cout << "Result is : " << sum << endl;
}

int main()
{
    add();
    return 0;
}
```

